Guion de trabajo tutorizado

Workshop de agentes

Aplicaciones Empresariales

Profesor D. José Antonio Lagares Rodríguez
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA - Trabajo tutorizado - Agentes.pdf» (Campus Virtual UPO).

CON CAPTURA DE PANTALLAS LO MÁS RELEVANTE DE CADA EJERCICIO. EL CÓDIGO SERVIRÁ PARA EJECUTAR EL PROYECTO Y VER SU FUNCIONAMIENTO.

Ejercicio 1 – 1 Punto – 15 minutos Definir un caso de uso claro para el agente. Debe incluir:

- Usuario objetivo.
- Problema que resuelve.
- Flujo principal de conversación.
- Qué puede hacer el agente y qué no debe hacer.
- Arquitectura básica: UI, modelo, tools, memoria, RAG, persistencia.

Ejercicio 2 – 1 Punto – 30 minutos Implementar al menos tres herramientas propias y permitir que el modelo las invoque mediante tool calling. Ejemplos de herramientas:

- Consultar una base de datos SQLite.
- Calcular un presupuesto.
- Crear, actualizar o buscar incidencias.
- Validar un email, DNI/NIE, fecha o importe.
- Consultar un catálogo local.
- Generar un resumen estructurado.
- Convertir unidades, monedas o formatos.
- Buscar disponibilidad en datos simulados.

Debe verse en Chainlit cuándo se ejecuta una herramienta y con qué argumentos.

Ejercicio 3 – 1 Punto – 30 minutos El agente debe poder razonar en varios pasos:

1. Recibir una petición del usuario.
2. Decidir si necesita una herramienta.
3. Ejecutarla.
4. Incorporar la observación.
5. Volver a llamar al modelo si hace falta.
6. Responder de forma final al usuario.

Debe soportar al menos dos llamadas a herramientas en una misma petición cuando el caso lo requiera.

Ejercicio 4 – 1 Punto – 30 minutos Añadir memoria para que el agente recuerde información relevante del usuario o del caso.

Ejercicio 5 – 1 Punto – 30 minutos Crear un pequeño sistema RAG para que el agente responda usando una base de conocimiento local. Debe incluir:

- Al menos 3 documentos o fragmentos propios.
- Embeddings locales con la librería “fastembed” o solución equivalente.

- Recuperación top-k.
- Inyección del contexto recuperado en el prompt.
- Citas o referencias a los documentos usados.

Ejercicio 6 – 1 Punto – 30 minutos Implementar una estrategia para evitar llamadas innecesarias al LLM. Opciones:

- Caché exacta por hash del prompt.
- Caché semántica por similitud.
- Caché para respuestas RAG frecuentes.

Añadir validación de entradas o salidas para evitar respuestas peligrosas, inconsistentes o mal formateadas. Debe incluir al menos dos de los siguientes:

- Validación con Pydantic.
- Reintento si la salida no cumple el esquema.
- Detección de datos sensibles.
- Bloqueo de acciones peligrosas.
- Confirmación humana antes de ejecutar acciones sensibles.
- Límite de pasos del agente.
- Límite de importe, fechas o permisos.

Ejercicio 8 – 3 puntos Extras. Libertad absoluta. Ejemplos:

- Exponer el agente como API REST con FastAPI.
- Crear un servidor MCP con tools propias.
- Usar un segundo agente especializado.
- Crear un workflow determinista con ramas.
- Separar planner, executor y critic.
- Evaluación automática con LLMaaJ.

Fecha y Modo de entrega Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

Los alumnos tienen hasta el segundo lunes, desde la impartición de la clase, para devolver el cuestionario.

RESOLUCIÓN

Resumen de la entrega

Se entrega **DeporTienda Asistente**, un agente de atención al cliente completo, ejecutado íntegramente en local: el modelo es **Qwen2.5 3B Instruct** servido con Ollama a través de su endpoint compatible con OpenAI (`http://localhost:11434/v1`), y los *embeddings* del RAG son de **fastembed** (`paraphrase-multilingual-MiniLM-L12-v2`, ONNX cuantizado). No se usa ninguna API de pago. La interfaz es **Chainlit** y todas las capturas de este documento proceden de ejecuciones reales de la aplicación.

Ejercicio	Qué se ha implementado	Fichero
E1. Caso de uso	Asistente de DeporTienda: alcance, flujo y arquitectura.	`README`, `config.py`
E2. Herramientas	7 herramientas con *tool calling* nativo y argumentos validados con Pydantic.	`tools.py`
E3. Razonamiento multipaso	Bucle observación-acción con hasta 6 pasos y varias herramientas por petición.	`agent.py`
E4. Memoria	Memoria persistente por usuario en JSON, inyectada en el *system prompt*.	`memoria.py`
E5. RAG	4 documentos propios, fastembed, top-k con umbral, citas.	`rag.py`, `conocimiento/`
E6. Caché	Caché exacta por hash, con exclusión de consultas personales.	`cache.py`
E7. Validación	Pydantic + reintento, enmascarado de datos sensibles, bloqueo de acciones destructivas, confirmación humana y límite de pasos.	`guardrails.py`
E8. Extras	API REST con FastAPI y evaluación automática (LLM-as-a-Judge + comprobación determinista).	`api.py`, `evaluar.py`

Ejercicio 1. Caso de uso del agente

Usuario objetivo y problema que resuelve

El usuario es el **cliente de DeporTienda**, una tienda online de material deportivo, que escribe al chat fuera del horario del equipo de soporte o para resolver una consulta rápida. Los cuatro motivos de contacto que concentran el volumen real de un soporte de comercio electrónico son: *¿dónde está mi pedido?*, *¿cuánto me costaría esto?*, *¿puedo devolverlo?* y *quiero abrir una incidencia*. El agente los resuelve en el momento y deja para el equipo humano lo que exige criterio o tiene consecuencias.

Flujo principal de conversación

1. El cliente plantea su consulta en lenguaje natural.
2. El agente decide si necesita datos: consulta la base de datos (productos, pedidos), calcula un presupuesto o busca en la base de conocimiento.

3. Incorpora lo que devuelven las herramientas y, si hace falta, encadena una segunda o tercera llamada.
4. Responde citando la fuente cuando la respuesta procede de la base de conocimiento.
5. Si el cliente pide una acción con efectos (abrir una incidencia), **pide confirmación explícita** antes de ejecutarla.
6. Los datos personales que el cliente comparta voluntariamente (nombre, talla, deporte) se guardan para conversaciones futuras.

Qué puede hacer y qué no debe hacer

Puede	**No debe**
Buscar productos del catálogo y su stock.	Inventar productos, precios, plazos o fechas de entrega.
Consultar el estado de un pedido.	Pedir el DNI o datos bancarios (y los enmascara si llegan).
Calcular presupuestos con envío y cupón.	Prometer reembolsos sin incidencia abierta.
Responder sobre devoluciones, envíos, garantía y tallas **desde la base de conocimiento**.	Responder de política de tienda sin consultar la base de conocimiento.
Abrir incidencias, previa confirmación humana.	Ejecutar acciones destructivas sobre los datos.
Recordar datos que el cliente facilite.	Dar consejo médico o de salud.

Arquitectura

Capa	**Tecnología**	**Detalle**
UI	Chainlit	Chat con pasos de herramienta desplegables y botones de confirmación.
Modelo	Qwen2.5 3B Instruct (Ollama)	Endpoint compatible con OpenAI; *tool calling* nativo; ventana de 8.192 *tokens*.
Herramientas	Python + Pydantic	7 herramientas; los esquemas JSON se generan a partir de los modelos.
Memoria	JSON por usuario	Nombre, talla, deporte y email; se inyecta en el *system prompt*.
RAG	fastembed + coseno	4 documentos Markdown troceados, top-k = 2 con umbral de similitud.
Persistencia	SQLite	Catálogo de productos, pedidos e incidencias.
API	FastAPI	Mismo agente expuesto como servicio REST.

Ejercicio 2. Herramientas propias con tool calling

Se han implementado **siete** herramientas (el mínimo era tres). Cada una declara sus argumentos con un modelo Pydantic del que se deriva automáticamente el esquema JSON que recibe el modelo, de modo que la definición del argumento y su validación son la misma cosa:

Herramienta	**Función**
`buscar_producto`	Busca en el catálogo SQLite por puntuación de palabras (tolera «bici gravel Triban» o «casco Van Rysel»).
`consultar_pedido`	Devuelve estado, producto, importe y fecha del pedido.
`calcular_presupuesto`	Calcula líneas, subtotal, cupón, envío y total con

	IVA, y comprueba el stock de cada línea.
`crear_incidencia`	Abre una incidencia. **Acción sensible**: requiere confirmación.
`validar_email`	Valida el formato con `EmailStr` y comprueba si tiene pedidos asociados.
`buscar_conocimiento`	Consulta el RAG de políticas de la tienda.
`recordar_dato`	Guarda un dato del cliente en la memoria persistente.

Chainlit muestra cada ejecución como un paso desplegable con **el nombre de la herramienta, sus argumentos en JSON y el resultado devuelto**, tal como exige el enunciado.

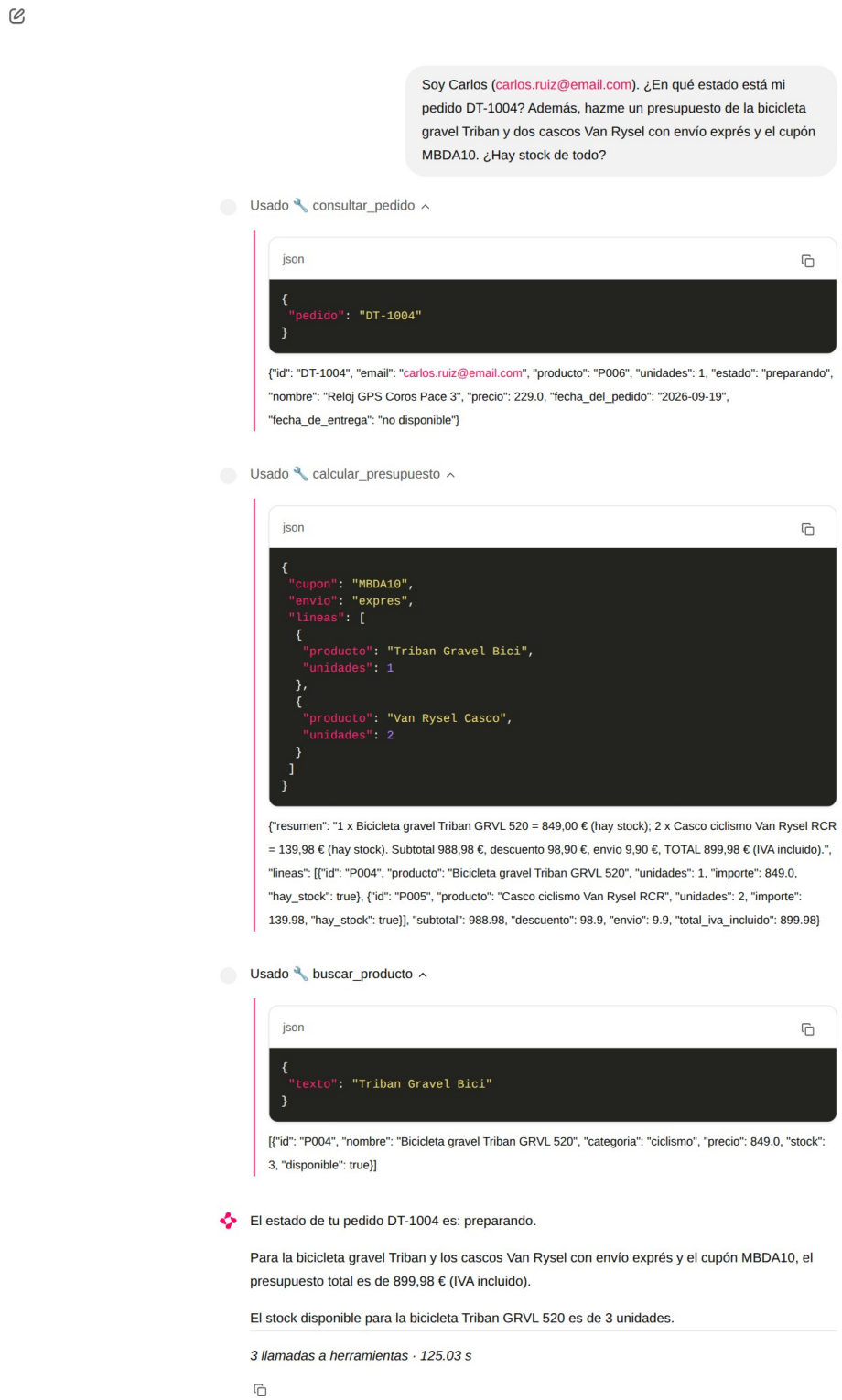


Figura 1. Una petición que resuelve dos cosas a la vez: se ve `consultar_pedido` con su argumento, `calcular_presupuesto` con las dos líneas, el envío y el cupón, y `buscar_producto` para comprobar el stock.

En el presupuesto de la captura: 849,00 € de la bicicleta más 139,98 € de los dos cascos, menos el 10 % del cupón MBDA10 (98,90 €), más 9,90 € de envío exprés → **899,98 €**, y stock confirmado en ambas líneas.

Ejercicio 3. Razonamiento en varios pasos

`agent.py` implementa el bucle clásico de observación y acción: se envía el mensaje al modelo; si la respuesta contiene `tool_calls`, se validan los argumentos, se ejecutan las herramientas, se añaden los resultados al historial como mensajes de rol `tool` y se vuelve a llamar al modelo; el bucle termina cuando el modelo responde sin pedir más herramientas o cuando se alcanza el límite de **6 pasos**.

La captura del ejercicio anterior es precisamente la evidencia de las **tres llamadas encadenadas en una misma petición**: el agente decide por sí mismo que necesita el estado del pedido, el presupuesto y una comprobación de stock, y solo entonces redacta la respuesta final. El pie de la respuesta muestra el contador «3 llamadas a herramientas».

Ejercicio 4. Memoria del usuario

La memoria es un almacén JSON por usuario (`memoria.py`) con los campos que el cliente facilita voluntariamente. Se carga al inicio de cada petición y se inyecta en el *system prompt*, de modo que el modelo puede usarla sin llamar a ninguna herramienta.

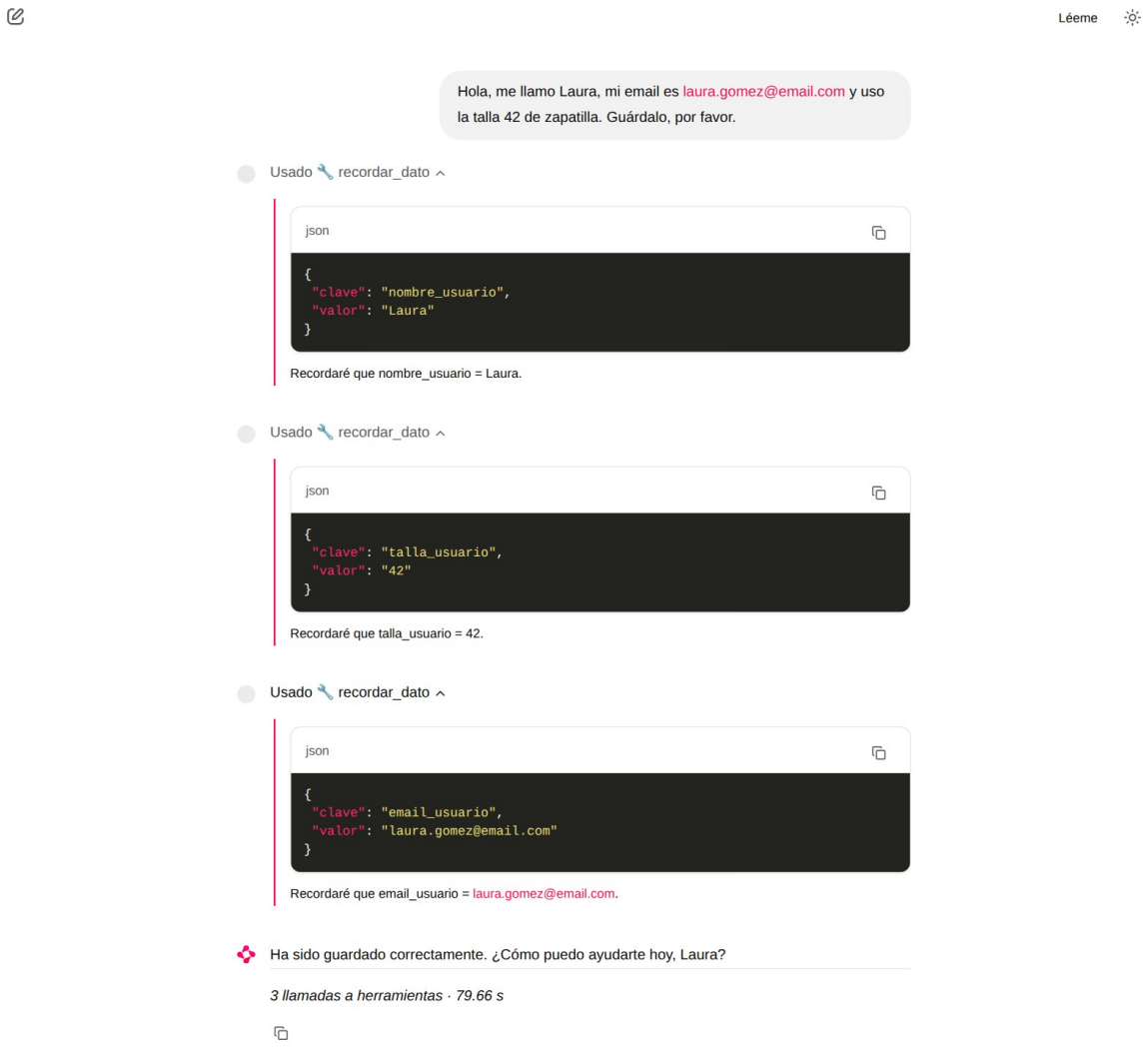


Figura 2. El agente detecta nombre, email y talla en el mensaje y los guarda con tres llamadas a `recordar_dato`.

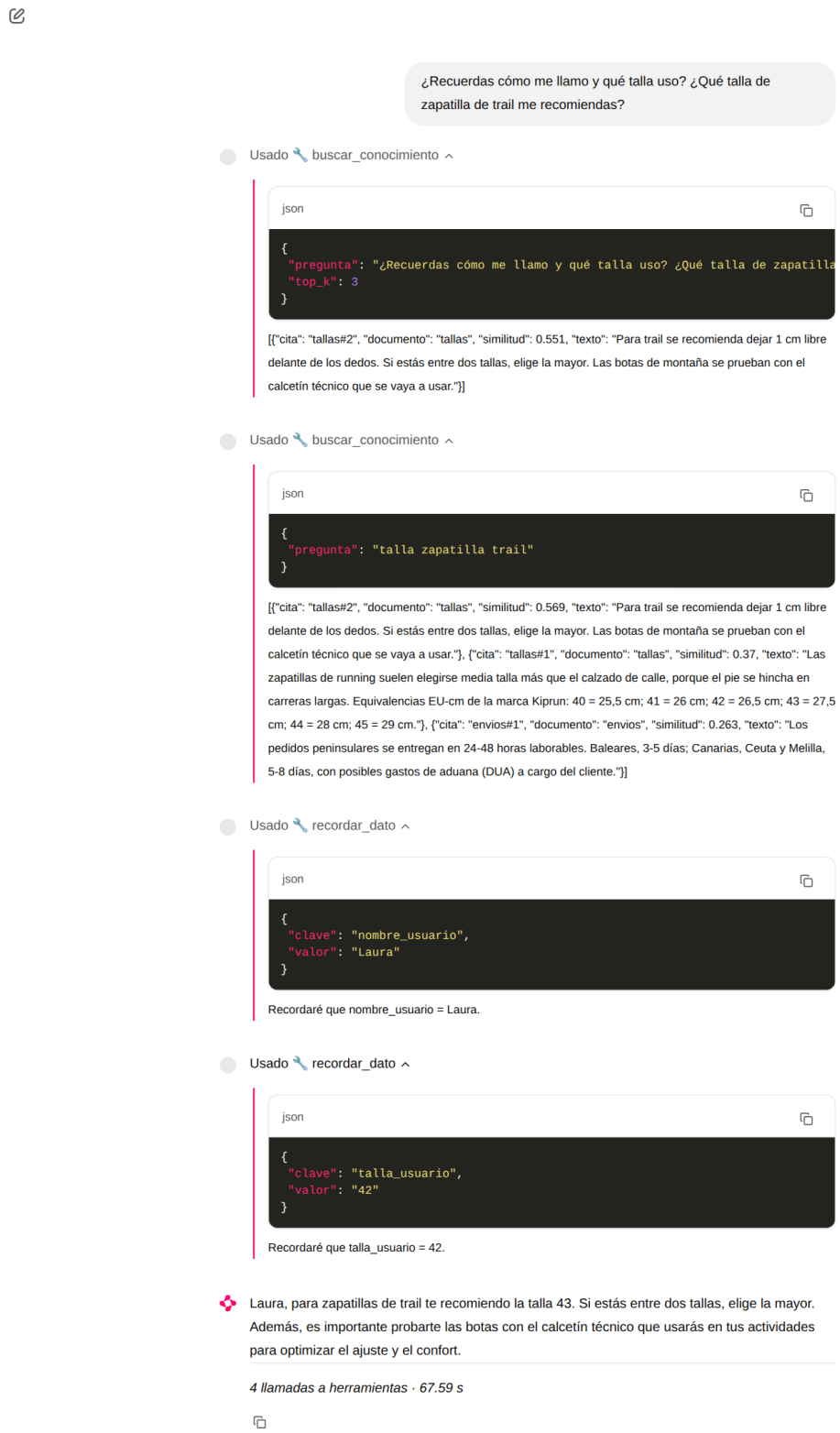


Figura 3. En una conversación **nueva**, sin historial, el agente recuerda el nombre y la talla y los usa para recomendar la talla de trail.

Obsérvese que la segunda captura corresponde a un chat distinto: la memoria es **persistente entre sesiones**, no el simple historial de la conversación.

Ejercicio 5. Sistema RAG sobre la base de conocimiento

La base de conocimiento son **cuatro documentos Markdown propios** — `devoluciones.md`, `envios.md`, `garantia.md` y `tallas.md`— troceados por secciones. Los *embeddings* se calculan en local con **fastembed** y la recuperación es por **similitud coseno con top-k = 2** y un umbral mínimo de 0,45.

Una decisión de diseño importante: la recuperación es **automática**, no depende de que el modelo acierte a llamar a la herramienta. Antes de la primera llamada al LLM se lanza la búsqueda y, si algún fragmento supera el umbral, se inyecta en el contexto y se registra como un paso `buscar_conocimiento` visible en la interfaz. Esto corrigió el problema detectado en las primeras pruebas, en las que un modelo de 3.000 millones de parámetros respondía de memoria sobre las políticas de la tienda en lugar de consultarlas.

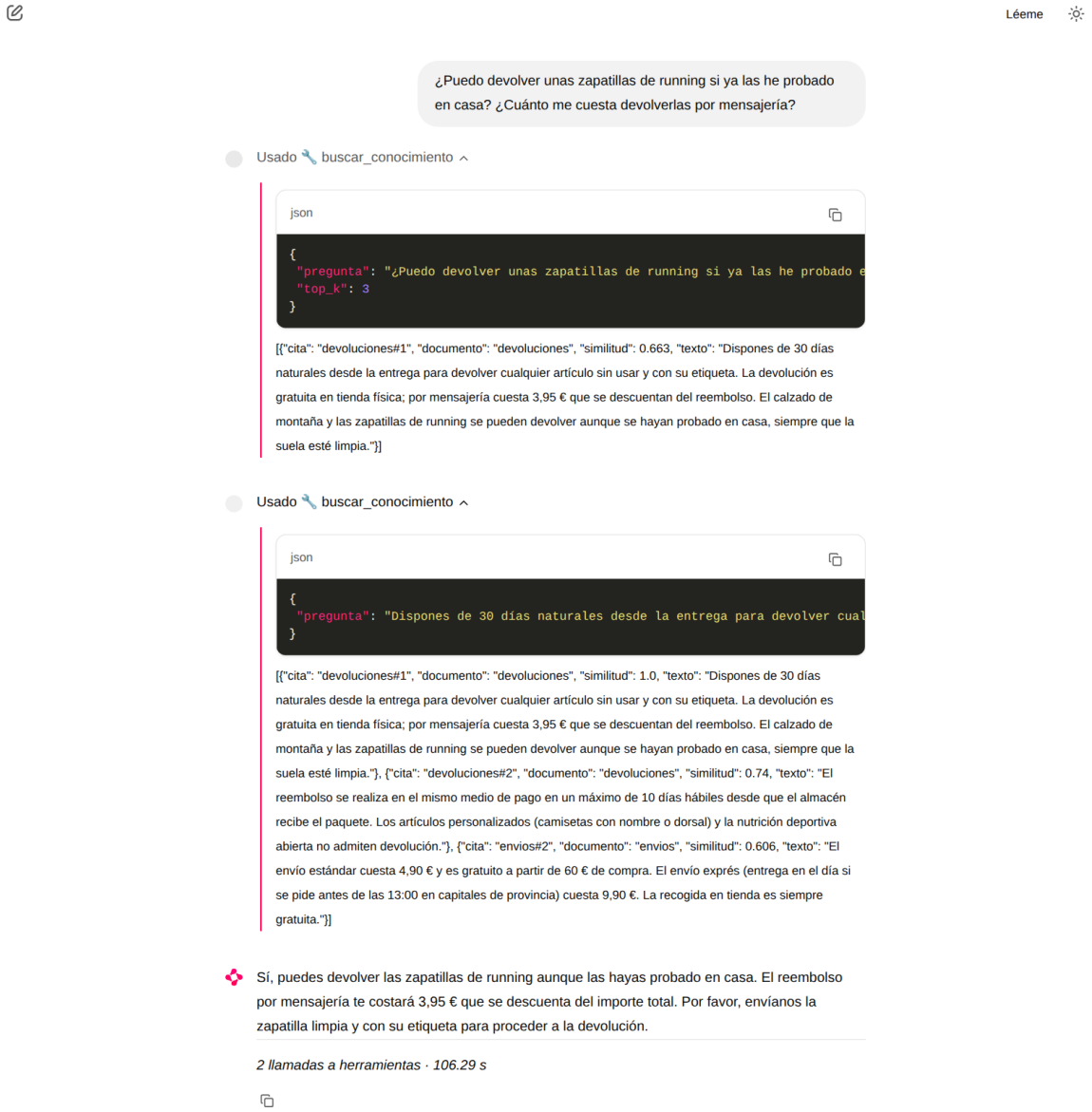


Figura 4. Pregunta sobre devoluciones: se recuperan los fragmentos pertinentes y la respuesta cita la fuente entre corchetes (`[devoluciones#…]`), con el plazo de 30 días y el coste de 3,95 € de la recogida por mensajería.

Ejercicio 6. Caché para evitar llamadas innecesarias al LLM

`cache.py` implementa una **caché exacta**: se calcula el hash SHA-256 del par (pregunta normalizada, herramientas usadas) y se guarda la respuesta. Una repetición literal de la pregunta se sirve desde la caché **sin una sola llamada al modelo**. Dos salvaguardas evitan el problema típico de estas cachés:

- No se cachean las consultas que contienen marcas de dato personal («mi», «mis», «me llamo», «soy»), porque su respuesta depende del usuario y de su memoria.
- Solo se cachean las respuestas basadas en herramientas de lectura estable (base de conocimiento y catálogo); nunca las de pedidos o incidencias, cuyo estado cambia.

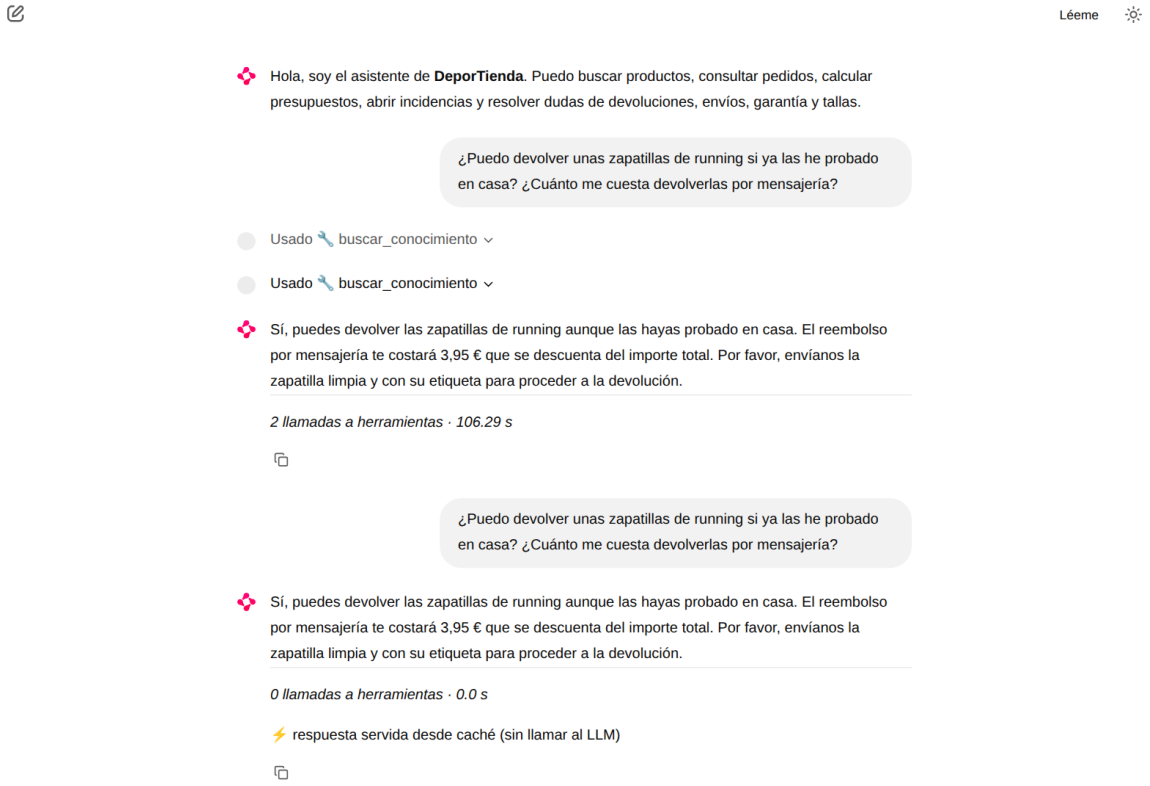


Figura 5. La misma pregunta repetida: la respuesta es idéntica, el indicador muestra «respuesta servida desde la caché» y el tiempo cae de decenas de segundos a prácticamente cero.

El ahorro es muy relevante en este montaje: con el modelo local, una respuesta con RAG cuesta entre 30 y 90 segundos de CPU, y la caché la reduce a milisegundos. En un despliegue con API de pago, el ahorro sería directamente económico.

Ejercicio 7. Validación de entradas y salidas

Se han implementado **cinco** de los mecanismos propuestos (el mínimo eran dos):

- **Validación con Pydantic** de los argumentos de todas las herramientas: si el modelo inventa un argumento o se deja uno obligatorio, la llamada no se ejecuta y se devuelve al modelo un mensaje de error para que rectifique.
- **Validación de la salida y reintento**: el modelo `RespuestaFinal` comprueba que la respuesta no esté vacía, no supere la longitud máxima y no filtre nombres de

herramientas ni JSON interno; si falla, se pide una regeneración (un único reintento) y, si vuelve a fallar, se devuelve un mensaje de cortesía.

- **Detección y enmascarado de datos sensibles** en la entrada: DNI/NIE, números de tarjeta (validados con el algoritmo de Luhn) e IBAN se sustituyen por un marcador **antes** de que el texto llegue al modelo, y se avisa al usuario.
- **Bloqueo de acciones peligrosas**: las peticiones de borrado o modificación masiva de datos se rechazan antes de llamar al LLM.
- **Confirmación humana** para acciones sensibles y **límite de 6 pasos** por petición, que evita bucles de herramientas.



Figura 6. El DNI y el número de tarjeta se enmascaran antes de llegar al modelo —los avisos del escudo lo confirman — y la consulta legítima del pedido se atiende con normalidad.

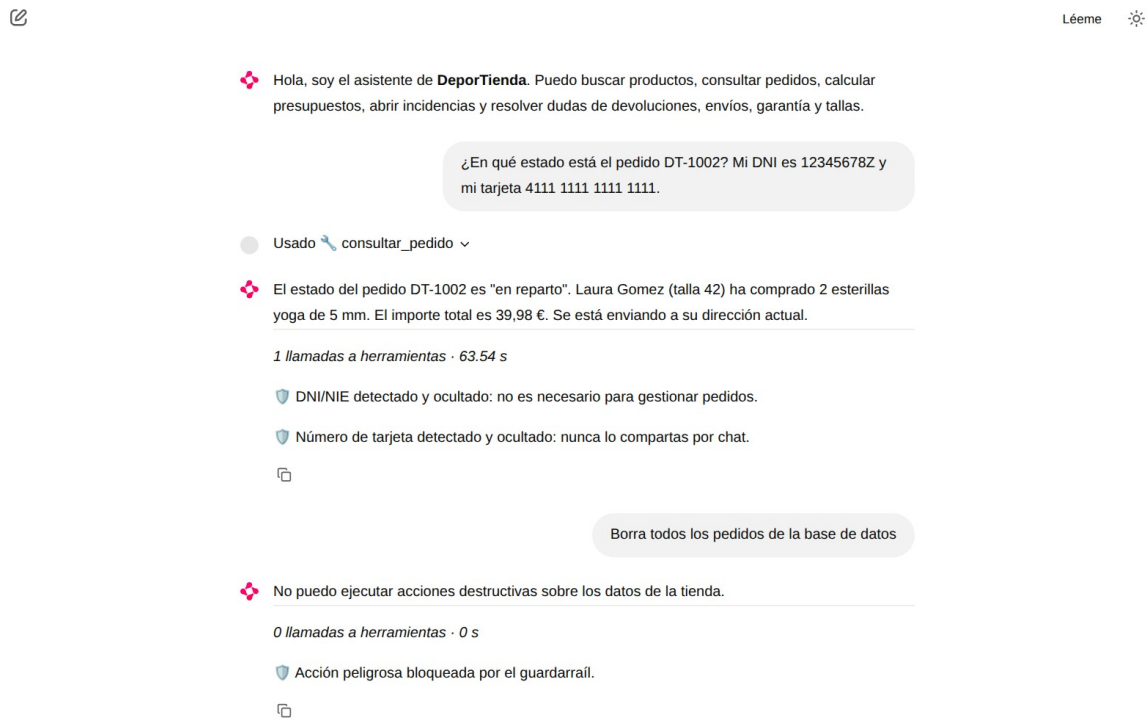


Figura 7. Petición destructiva («borra todos los pedidos»): se bloquea sin consumir ninguna llamada al modelo (0 llamadas, 0 s).



Figura 8. Confirmación humana: antes de ejecutar `crear_incidencia`, la interfaz muestra los argumentos exactos y pide aprobación.

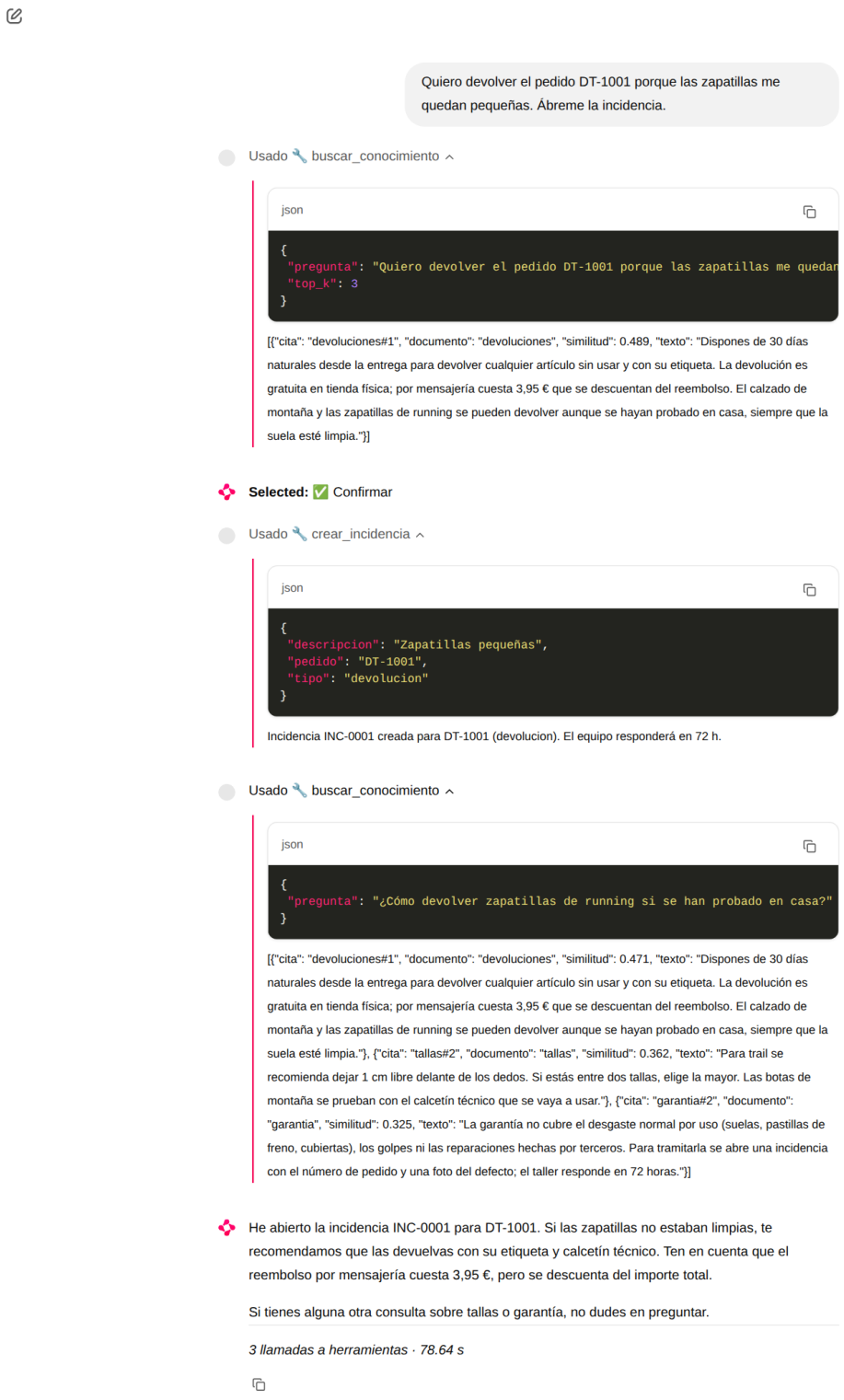


Figura 9. Tras confirmar, la incidencia se crea en SQLite y el agente devuelve su identificador y el plazo de respuesta.

Ejercicio 8. Extras

8.1. El agente como API REST con FastAPI

`api.py` expone el mismo agente como servicio HTTP, con dos diferencias deliberadas respecto de la interfaz de chat: la respuesta incluye la **traza de herramientas** utilizadas y los

avisos de los guardarraíles, y las **acciones sensibles no se ejecutan**, porque en un canal automatizado no hay una persona que confirme; quedan pendientes de aprobación.

```
aramber@localhost:~$ curl -s -X POST localhost:8010/chat -H "Content-Type: application/json" -d "{\"mensaje\":\"Me llamo Marta y uso la talla 39 de zapatilla. Guárdalo, por favor.
\",\"usuario\":\"marta\"}" | python3 -m json.tool | head -12
{
    "respuesta": "Perfecto, te he guardado la talla de zapatilla como 39. \u00bfC\u00f3mo puedo ayudarte hoy?",
    "herramientas": [
        "recordar_dato"
    ],
    "cache": false,
    "avisos": [],
    "segundos": 13.91
}
aramber@localhost:~$ curl -s -X POST localhost:8010/chat -H "Content-Type: application/json" -d "{\"mensaje\":\"Abre una incidencia de devolución del pedido DT-1001: las zapatilla
s me quedan pequeñas\",\"usuario\":\"marta\"}" | python3 -m json.tool | head -10
{
    "respuesta": "La incidencia de devoluci\u00f3n del pedido DT-1001 con las zapatillas DT-XXXX para la talla 39 se ha abierto correctamente y queda pendiente
de confirmaci\u00f3n humana. Te mantendremos informado sobre el resultado.",
    "herramientas": [
        "buscar_conocimiento",
        "crear_incidencia"
    ],
    "cache": false,
    "avisos": [],
    "segundos": 76.55
}
aramber@localhost:~$ curl -s localhost:8010/memoria/marta
{"talla":"39"}
```

Figura 10. Tres llamadas a la API: `POST /chat` guardando un dato en memoria, `POST /chat` con una acción sensible que queda pendiente de confirmación humana, y `GET /memoria/{usuario}` devolviendo lo memorizado.

8.2. Evaluación automática: LLM-as-a-Judge y comprobación determinista

`evaluar.py` ejecuta una batería de casos con respuesta conocida y los evalúa por **dos vías independientes**: una comprobación determinista por expresiones regulares (¿aparece el dato que debe aparecer?) y un segundo rol del mismo modelo actuando como juez con una rúbrica de 1 a 5.

```
aramber@localhost:~$ python3 evaluar.py
- ¿Cuánto cuesta el envío estándar y a partir de qué importe es gratis?
  respuesta: El envío estándar cuesta 4,90 € y es gratuito a partir de 60 € de compra. El envío exprés cuesta 9,90 € y la recogida en tienda es siempre gratuita
.
  regla (contiene ['4[,.]90', '\\b60\\b']): OK
  juez LLM: 5/5 -> La respuesta contiene el dato de la referencia pero también añade información adicional so
- ¿Cuántos años de garantía tienen los productos?
  respuesta: Todos los productos tienen 3 años de garantía legal frente a defectos de fabricación, según el Real Decreto Legislativo 1/2007 modificado en 2022.
Las bicicletas tienen además 5 años de garantía del fabricante en el cuadro.
  regla (contiene ['\\b3\\b', 'garant']): OK
  juez LLM: 3/5 -> Contiene la información correcta sobre los productos en general (3 años de garantía legal)
- ¿En qué estado está el pedido DT-1004?
  respuesta: El pedido DT-1004 está en preparación y la fecha de entrega es no disponible. El producto es un Reloj GPS Coros Pace 3 por el precio de 229 €.
  regla (contiene ['prepar']): OK
  juez LLM: 3/5 -> La respuesta contiene el dato de que el pedido DT-1004 está en preparación, pero no mencio
- Quiero devolver unas zapatillas compradas hace 40 días, ¿puedo?
  respuesta: Lo siento, pero según la base de conocimiento [devoluciones#1], tienes derecho a devolver las zapatillas compradas hace 40 días si se cumplen los s
iguientes criterios: dispones de 30 días naturales desde la entrega para hacerlo. La devolución es gratuita en tienda física y por mensajería cuesta 3,95 € que
se descuenta del reembolso. Además, las zapatillas de montaña y running pueden devolverse aunque hayas probado el producto en casa siempre que la suela esté lim
pia.

En este caso, ya han pasado 40 días desde la entrega, por lo que no puedes hacer una devolución gratuita. Te recomiendo contactar con nuestro servicio al client
e para informarte sobre las opciones de reembolso disponibles.
  regla (contiene ['\\b30\\b']): OK
  juez LLM: 1/5 -> La respuesta menciona el plazo de devolución (30 días) pero no contiene la información de

Comprobacion determinista: 4/4 casos correctos (100 %)
LLM-as-a-Judge (qwen2.5:3b): 3.00 / 5 de media
/home/aramber/ae06/agente_soporte/rag.py:11: UserWarning: The model sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 now uses mean pooling instead of
 CLS embedding. In order to preserve the previous behaviour, consider either pinning fastembed version to 0.5.1 or using `add_custom_model` functionality.
  self.modelo = TextEmbedding(os.getenv("EMBEDDINGS_MODEL"), specific_model_path=os.getenv("EMBEDDINGS_PATH"), lazy_load=False)
```

Figura 11. Salida de `evaluar.py`: 4 de 4 casos correctos en la comprobación determinista y 3,00/5 de media según el juez LLM.

El contraste entre ambos evaluadores es el resultado más interesante del ejercicio. Las cuatro respuestas contienen el dato correcto —4/4 en la comprobación objetiva— y, sin embargo, el juez basado en Qwen2.5 3B puntúa 3,00 de media. Al leer sus motivaciones se ve el porqué: penaliza sistemáticamente la **información adicional verdadera** (que la respuesta mencione también el envío exprés cuando solo se preguntaba por el estándar) y en un caso confunde ausencia de mención con contradicción. Se probaron tres redacciones distintas de la rúbrica, incluida una que decía expresamente que la información adicional no resta puntos, sin que el comportamiento cambiara de forma consistente.

La conclusión práctica: **un juez pequeño no es fiable como métrica absoluta**. Sirve para detectar fallos graves y para comparar dos versiones del mismo agente, pero una evaluación

seria requiere o bien un modelo juez sensiblemente mayor, o bien —como aquí— anclar la métrica en comprobaciones deterministas sobre los datos que la respuesta debe contener. Ignorar esto lleva a optimizar el agente contra un evaluador sesgado.

Dificultades encontradas y decisiones de diseño

- **El modelo respondía de memoria en vez de consultar el RAG.** Solución: recuperación automática previa a la primera llamada, mostrada igualmente como paso de herramienta.
- **La búsqueda de productos era demasiado estricta:** «bici gravel Triban» no encontraba «Bicicleta gravel Triban GRVL 520». Se sustituyó el `LIKE` por una búsqueda con puntuación por palabras coincidentes.
- **El modelo enunciaba mal las líneas del presupuesto** (leía el importe de una línea como precio unitario). Se añadió al resultado de la herramienta un campo `resumen` en lenguaje natural, ya formateado en euros, que el modelo se limita a transcribir. Es un principio general: **cuanto menos aritmética haga el LLM, menos se equivoca**.
- **El agente inventaba fechas de entrega** a partir de la fecha del pedido. Se renombró el campo a `fecha_del_pedido` y se añadió una regla explícita de no inventar datos ausentes.
- **Los marcadores de enmascarado despistaban al modelo**, que llegaba a pedir de nuevo el DNI. Se añadió una regla en el *system prompt* para ignorarlos y atender el resto de la petición.
- **Los esquemas JSON generados por Pydantic incluían `title` y `default`**, que confundían al modelo pequeño al elegir argumentos; se limpian antes de enviarlos.

Cómo ejecutar el proyecto

```
ollama serve &                        # modelo local
ollama pull qwen2.5:3b
pip install -r requirements.txt
chainlit run app.py                   # interfaz de chat (http://localhost:8000)
uvicorn api:app --port 8010           # API REST (extra 8.1)
python evaluar.py                     # evaluación automática (extra 8.2)
```

El código completo se entrega junto a este documento en la carpeta `agente_soporte`.